

A Comprehensive Stock Market Database for Real-Time Analysis and Transaction Management

Taran Mamidala
(50604177)
Computer Science and Engineering
University at Buffalo
New York, USA
taranmam@buffalo.edu

Praanav Bhowmik
(50594516)
Computer science and Engineering
University at Buffalo
New York, USA
praanavb@buffalo.edu

Rachabattuni Sai Shashi
Kiran(50602502)
Computer science and Engineering
University at Buffalo
New York, USA
srachaba@buffalo.edu

I. PROBLEM STATEMENT

In today's unstable financial arrangements, the intricacy and sheer volumes of stock market data have surpassed traditional tools such as Excel. The management of such data with millions of history of prices, transactions, company moves, market news, and broker analysts' forecasts is disabled as well as flawed in comparison to the management of such data. The objective of our project is to design a solid SQL-based database that normalizes and aggregates stock market data to ensure expert-level queries, real-time updates and data integrity. The project domain is the stock market domain. This is an extremely suitable domain, as it is actual-world financial systems used by traders, analysts and trading platforms. The primary goal is to build a solid database that can handle complex analytical queries (eg, past performance analysis, trend analysis and portfolio presentation) and transaction updates (eg, new user sign-up, stock trades and corporate

II. BACKGROUND AND SIGNIFICANCE

A. Background of the Problem:

The financial marketplace produces enormous quantities of everyday info, including historic costs, trading volumes, financial statements and corporate announcements. Data analysts have historically utilized Excel spreadsheets and independent flat files to crunch. But as the size and intricacy of the dataset has increased, such devices have turned into impossibilities. Excels possess scalability, power and incapability of imposing data integrity on associated datasets.

Our aim stemmed from this necessity. There is a great demand in the contemporary dynamic markets for a centralized, generalized database that not only stores large volumes of financial data, but also supports real-time queries and updates. Stock market demands the interconnectivity and sophistication of data, an association with advanced querial capability, strict data integrity and efficient data processing.

B. Potential Contribution to the Problem Domain:

Our Market Intelligence Database project contributes significantly to the problem domain by:

x

- **Improving Data Accessibility and Analysis:**
By consolidating the diverse dataset in a single, generalized database, our solution provides an integrated platform for detailed financial analysis. Analysts can re-recover historical trends, analyze market events and make data-managed investment decisions.
- **Enhancing Data Integrity and Performance:**
Clearly defined primary and rigid schema design (in BCNF) ensures that the data remains consistent and reliable. This integrity is essential for important financial decisions and regulatory compliance.
- **Supporting real -time updates and scalability:**
The database is designed to handle continuous data updates and skilled efficiently with data volume growing. This adaptability is important in an environment where the market situation changes rapidly.
- **Advanced analysis and automation enabling:**
With its strong structure, the database gives groundwork for future enhancement such as predictive analytics, automatic trading algorithms and real -time dashboard.

C. WHY A DATABASE INSTEAD OF EXCEL?

Scalability: SQL database can manage millions of records efficiently, while Excel slows down with large datasets and becomes error-damaged.

Data Integrity: Conflict database applies data stability with obstacles (primary keys, foreign keys, check obstacles) that cannot give excel guarantee.

Querying Power: SQL provides the necessary operations, aggregation and analytical functions for deep data insight.

Multi-User Concurrency: Database supports several concurrent users and strong safety measures, which are essential for the production environment.

III. TARGET USERS

A. Primary Users:

Financial Analysts and Investors:

These users require rapid access to historical and real-time data to analyze market trends, compare stock performance and inform investment decisions. Our database will provide detailed transactions to the depth analysis to provide detailed transactions, performance metrics and corporate functions.

Database Administrators:

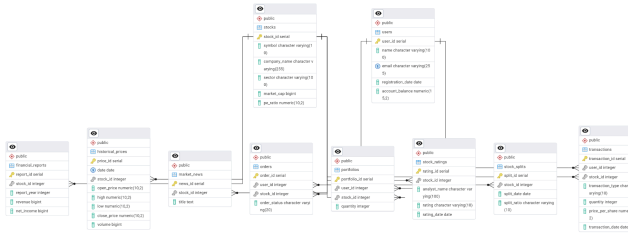
IT and Database Professionals:

Responsible for maintaining the database, ensuring data security, backuping and optimizing performance. They will use our system to manage regular data updates, monitor system health and support user questions.

B. Real-Life Scenario:

Imagine a financial advisory firm where analysts use the market intelligence database to draw historical value trends, review corporate tasks and monitor real-time market news. Portfolio managers use user transactions and portfolio data to accommodate investment strategies. In addition, the DBA ensures that the system is up-to-date, safe and responsible, supporting a multi-user environment with both analytical and transactions.

IV. E/R DIAGRAM AND SCHEMA DESCRIPTION



Our E/R diagram models 10 core entities that encapsulate the diverse aspects of the stock market.

A. E/R Diagram Overview

Our ER diagram comprises 10 key entities:

1. **Stocks** : Central repository for stock details.
2. **Historical_Prices** : Stores daily price data with a composite unique key on (date, stock_id).
3. **Users** : Contains user account details.
4. **Transactions** : Records each buy/sell transaction by users.
5. **Portfolios** : Represents user holdings of stocks.
6. **Orders** : Tracks orders (pending, executed, or cancelled) placed by users.
7. **Market_News** : Captures news articles related to stocks.
8. **Financial_Reports** : Contains annual financial reports.
9. **Stock_Splits** : Records historical stock split events.
10. **Stock_Ratings** : Stores analyst ratings for stocks.

Each relationship is enforced with **primary and foreign key constraints**, ensuring data integrity across the database.

B. Relationships

Stocks:

Acts as the central entity. It is referenced by historical_prices, market_news, financial_reports, stock_splits, and stock_ratings. Additionally, transactions, portfolios, and orders reference stocks.:

- **One-to-many** with Historical_Prices, Market_News, Financial_Reports, Stock_Splits, and Stock_Ratings.
- **Referenced by** Transactions, Portfolios, and Orders.

Users: Holds user details and is referenced by transactions, portfolios, and orders.

- **One-to-many** with Transactions, Portfolios, and Orders.

Historical_Prices uses a **composite key** (date, stock_id) ensuring that each stock has one record per trading day.

C. Detailed Relation Descriptions

For each table, we define attributes, data types, primary keys, foreign keys, and constraints. For example:

Stocks Table

Attributes:

- **stock_id (INTEGER, PK):** A unique identifier for each stock.
- **symbol (VARCHAR(10), NOT NULL):** The stock symbol (e.g., AAPL, MSFT).
- **company_name (VARCHAR(255)):** The full name of the company.
- **sector (VARCHAR(100)):** The industry sector to which the company belongs (e.g., Technology).
- **market_cap (BIGINT):** The market capitalization, representing the total market value of the company's outstanding shares.
- **pe_ratio (DECIMAL(10,2)):** The Price-to-Earnings ratio, indicating the company's valuation relative to its earnings.

Purpose:

To uniquely identify each stock and store its basic information, which serves as the foundation for linking with all other tables in the database.

Historical_Prices Table

Attributes:

- **price_id (SERIAL, PK):** A surrogate key that uniquely identifies each historical price record.
- **date (DATE, NOT NULL):** The trading date for the stock price data.
- **stock_id (INT, NOT NULL, FK → stocks):** References the stock_id in the Stocks table to associate the price record with a specific stock.
- **open_price (DECIMAL(10,2)):** The opening price of the stock on that date.

- **high (DECIMAL(10,2))**: The highest price reached during the trading day.
- **low (DECIMAL(10,2))**: The lowest price recorded during the day.
- **close_price (DECIMAL(10,2))**: The closing price of the stock at the end of the day.
- **volume (BIGINT)**: The number of shares traded on that day.

Purpose:

To store detailed daily price data for each stock, supporting historical analysis and trend forecasting.

Foreign Key Action:

When a stock record is deleted from the Stocks table, all related historical price records are automatically removed (ON DELETE CASCADE).

Users Table

- **Attributes:**
 - **user_id (INTEGER, PK)**: a unique identifier for each user.
 - **name (VARCHAR(100))**: User Name.
 - **email (VARCHAR(255), UNIQUE, NOT NULL)**: The user's email address, which must be unique and necessary.
 - **registration_date (DATE, NOT NULL)**: The date on which the user was registered.
 - **account_balance (DECIMAL(15,2))**: current balance in the user's account.

Purpose:

To manage the user information required to track transactions, portfolio and order.

Transactions Table

- **Attributes:**
 - **transaction_id (INTEGER, PK)**: A unique identifier for each transaction.
 - **user_id (INT, NOT NULL, FK → users)**: The user refers to user_id in the table.
 - **stock_id (INT, NOT NULL, FK → stocks)**: Reference of Stock_ID in stock table.
 - **transaction_type (VARCHAR(10))**: The permissible values are 'buy' or 'sell'.
 - **quantity (INTEGER)**: The number of trading shares (should be more than 0).
 - **price_per_share (DECIMAL(10,2))**: The value at which each stock was traded.
 - **transaction_date (DATE, NOT NULL)**: that date when transactions occur.

Purpose:

To facilitate financial analysis and transaction tracking, to provide financial analysis and transaction tracking.

Foreign Key Actions:

On removal of user or stock, related transactions are automatically removed (on delete cascade)

Portfolios Table

- **Attributes:**
 - **portfolio_id (INTEGER, PK)**: A unique identifier for each portfolio record.
 - **user_id (INT, NOT NULL, FK → users)**: References the user_id in the Users table.
 - **stock_id (INT, NOT NULL, FK → stocks)**: References the stock_id in the Stocks table.
 - **quantity (INTEGER)**: The number of shares held by the user (must be greater than 0).

Purpose:

To represent the current holdings of each user, enabling portfolio tracking and valuation.

Foreign Key Actions:

Deletion of a user or stock cascades to remove associated portfolio records (ON DELETE CASCADE).

Orders Table

- **Attributes:**
 - **order_id (INTEGER, PK)**: This is a unique identifier for each order we handle.
 - **user_id (INT, NOT NULL, FK → users)**: This connects to the user_id in the Users table, allowing us to track who placed the order.
 - **stock_id (INT, NOT NULL, FK → stocks)**: This links to the stock_id in the Stocks table, showing which stock corresponds to the order.
 - **order_status (VARCHAR(20))**: This reflects the current status of the order, with options like 'PENDING', 'EXECUTED', or 'CANCELLED'.

Purpose:

This reflects the current status of the order, with options like 'PENDING', 'EXECUTED', or 'CANCELLED'.

Foreign Key Actions:

If a user or stock is deleted, all related orders will also be automatically removed (ON DELETE CASCADE).

Market_News Table

- **Attributes:**
 - **news_id (INTEGER, PK)**: This serves as a unique identifier for each news entry..
 - **stock_id (INT, NOT NULL, FK → stocks)**: This connects to the stock_id in the Stocks table
 - **title (TEXT, NOT NULL)**: This represents the title or headline of the news article.

Purpose:

The aim here is to retain news articles or market updates

that are pertinent to a stock, providing context for market movements.

Foreign Key Actions:

If a stock is deleted, any associated news entries will be automatically removed (ON DELETE CASCADE).

Financial_Reports Table

Attributes:

- **report_id (INTEGER, PK):** This serves as a unique ID for each report.
- **stock_id (INT, NOT NULL, FK → stocks):** This connects to the stock_id found in the Stocks table.
- **report_year (INTEGER, CHECK (report_year BETWEEN 2000 AND 2024)):** This specifies the year the report was created.
- **revenue (BIGINT):** This indicates the total annual revenue of the company.
- **net_income (BIGINT):** This reflects the company's net income.

Purpose:

The aim here is to capture the annual financial performance, which is essential for performing fundamental analysis.

Foreign Key Actions:

If a stock record is deleted, all related financial reports will be deleted as well (ON DELETE CASCADE).

Stock_Splits Table

Attributes:

- **split_id (INTEGER, PK):** This is a unique identifier for each stock split event.
- **stock_id (INT, NOT NULL, FK → stocks):** This connects to the stock_id in the Stocks table.
- **split_date (DATE, NOT NULL):** This records the date when the stock split occurred.
- **split_ratio (VARCHAR(10), NOT NULL):** This shows the split ratio (for instance, "2:1").

Purpose:

The aim here is to monitor corporate actions related to stock splits, as these can influence share prices and the holdings of investors.

Foreign Key Actions:

If a stock is deleted, any associated split records will be automatically removed (ON DELETE CASCADE).

Stock_Ratings Table

Attributes:

- **rating_id (INTEGER, PK):** This is a unique identifier for each rating record, ensuring that every entry stands out on its own.
- **stock_id (INT, NOT NULL, FK → stocks):** This connects to the stock_id in the Stocks table, helping us identify which stock the rating is associated with.
- **analyst_name (VARCHAR(100)):** In this field, we store the name of the analyst who issued the rating.
- **rating (VARCHAR(10), CHECK (rating IN ('BUY', 'HOLD', 'SELL'))):** This section captures the analyst's recommendation, whether it's a 'BUY', 'HOLD', or 'SELL'.
- **rating_date (DATE, NOT NULL):** This keeps track of the date when the rating was given.

Purpose:

The primary aim here is to collect and analyze the insights of analysts, as their opinions can greatly influence investor choices.

Foreign Key Actions:

If a stock is deleted, all associated rating records will be automatically removed as well (ON DELETE CASCADE).

TABLE I. DESCRIPTION OF ALL SCHEMAS

Table Name	Primary Key	Foreign Keys	Key Attributes
Stocks	stock_id (INTEGER)	None	stock_id, symbol, company_name, sector, market_cap, pe_ratio
Historical_Prices	price_id (SERIAL)	stock_id (references stocks(stock_id))	price_id, date, stock_id, open_price, high, low, close_price, volume; Composite unique key: (date, stock_id)
Users	user_id (INTEGER)	None	user_id, name, email, registration_date, account_balance
Transactions	transaction_id (INTEGER)	user_id (references users(user_id)); stock_id (references stocks(stock_id))	transaction_id, user_id, stock_id, transaction_type, quantity, price_per_share, transaction_date
Portfolios	portfolio_id (INTEGER)	user_id (references users(user_id)); stock_id (references stocks(stock_id))	portfolio_id, user_id, stock_id, quantity; Composite unique key: (user_id, stock_id)

Orders	order_id (INTEGER)	user_id (references users(user_id)); stock_id (references stocks(stock_id))	order_id, user_id, stock_id, order_status
Market_News	news_id (INTEGER)	stock_id (references stocks(stock_id))	news_id, stock_id, title
Financial_Reports	report_id (INTEGER)	stock_id (references stocks(stock_id))	report_id, stock_id, report_year, revenue, net_income
Stock_Splits	split_id (INTEGER)	stock_id (references stocks(stock_id))	split_id, stock_id, split_date, split_ratio
Stock_Ratings	rating_id (INTEGER)	stock_id (references stocks(stock_id))	rating_id, stock_id, analyst_name, rating, rating_date

V. IMPLEMENTATION

A. Dataset Acquisition and Transformation:

Real Data: We obtained real-time historical prices for 10 well-known stock symbols using Yahoo Finance.

Synthetic Data: We generated some more synthetic data to ensure that each table contains 3,000 to 6,000 rows. We maintained proper foreign key relationships by referencing valid stock and user IDs.

Data Cleansing: We handled missing values through forward- and backward-fill operations in our Python scripts. Historical Prices data particularly was carefully processed to eliminate all inconsistencies, for example, representation of decimal points in the volume field.

B. BCNF and Schema Explanation:

Functional Dependencies:

All the non-key columns in all the tables are completely dependent on the primary key alone. For example, stock_id in stocks table determines symbol, company_name, sector, market_cap, and pe_ratio.

No Redundancies:

Our implementation eliminates all the transitive and partial dependencies. In table historical_prices, composite key (stock_id, date) will be utilized to identify price data of a stock for a day.

Referential Integrity:

All foreign keys are constrained with ON DELETE CASCADE, so that deleting a parent row (say, a stock) will automatically delete all corresponding records in child tables.

BCNF Compliance:

Every single table in our schema is BCNF. If any relation had a candidate key which was not determining each non-key attribute, we would have decomposed it. In our design, however, all relations satisfy the conditions for BCNF.

VI. SQL QUERIES TO TEST DATABASE

Here in this task we are inserting, deleting, updating, and SELECT operations on the stock trading database through the operational queries. The select queries include JOIN by the use of ORDER BY, GROUP BY, and SUBQUERIES. Then, we have written stored procedures and functions to methodically execute common operations. We have tested all the queries, and the execution screenshots are presented.

1. Inserting.

Inserting the Test record

stock_id [PK] integer	symbol character varying (10)	company_name character varying (255)	sector character varying (100)	market_cap bigint	pe_ratio numeric (10,2)
3213	GOOGL	Company GOOGL 3213	Technology	588399172480	46.87
3214	AMZN	Company AMZN 3214	Technology	210321851362	18.50
3215	TSLA	Company TSLA 3215	Technology	405332424195	14.81
3216	META	Company META 3216	Technology	448435960382	37.71
3217	NFLX	Company NFLX 3217	Technology	885184724848	41.74
3218	TEST	TestCo, Inc.	Utilities	500000000	17.33

2. Deleting

Deleting the order records older than Jan 1 20225

```

10 -- DELETE to remove any Cancelled orders older than Jan 1 2025
11 DELETE FROM orders
12 WHERE order_status = 'CANCELLED'
13    AND order_id IN (
14        SELECT order_id
15        FROM orders

```

Data Output Messages Notifications

DELETE 1270

Query returned successfully in 122 msec.

3. Update PE Ratios for Utility Sector

```

20 -- UPDATE to bump up PE ratios by 10% for all utility stocks
21 UPDATE stocks
22 SET pe_ratio = pe_ratio * 1.10
23 WHERE sector = 'Utilities';

```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 152 msec.

4. Update User Account Balances

```
28 -- UPDATE to increase account balance by $100 for every user with a portfolio
29 UPDATE users
30 SET account_balance = account_balance + 100
31 WHERE user_id IN (SELECT DISTINCT user_id FROM portfolios);
32
```

Data Output Messages Notifications

	user_id [PK] integer	name character varying (100)	email character varying (255)	registration_date date	account_balance numeric (15,2)
1	1	User_1	user_1@example.com	2023-01-12	6786.96
2	2	User_2	user_2@example.com	2022-08-18	9648.37
3	3	User_3	user_3@example.com	2024-08-14	4323.10
4	4	User_4	user_4@example.com	2023-06-11	7817.05
5	5	User_5	user_5@example.com	2023-08-25	5326.14
6	6	User_6	user_6@example.com	2024-10-07	1607.00

5. List Open Orders with Stock Symbols (JOIN + ORDER BY)

Data Output Messages Notifications

	user_name character varying (100)	order_id integer	ticker character varying (10)	order_status character varying (20)
1	User_772	3986	NFLX	PENDING
2	User_3113	3982	AAPL	PENDING
3	User_839	3981	AMZN	PENDING
4	User_1471	3976	TSLA	PENDING
5	User_2124	3975	META	PENDING
6	User_2072	3972	META	PENDING

Total rows: 1344 Query complete 00:00:00.211

6. Total Shares Held per Sector (GROUP BY)

Data Output Messages Notifications

	sector character varying (100)	total_shares bigint
1	Technology	964644
2	Communication Services	1885
3	Consumer Cyclical	1065

7. Find High-Spending Users (Subquery)

Data Output Messages Notifications

	name character varying (100)	email character varying (255)
1	User_9	user_9@example.com
2	User_11	user_11@example.com
3	User_14	user_14@example.com
4	User_17	user_17@example.com
5	User_23	user_23@example.com
6	User_25	user_25@example.com

8. Stocks with More Than 5 "Buy" Ratings (JOIN + GROUP BY + HAVING)

Data Output Messages Notifications

	symbol character varying (10)	buy_count bigint
1	NVDA	146
2	TSLA	169
3	INTC	176
4	AAPL	165
5	GOOGL	156
6	META	154

VII. TRANSACTIONS AND TRIGGERS

Taking a thoughtful, critical approach to Database systems is non-negotiable in failure of transaction integrity and thus proper error reporting. Here, we have modeled a transaction that would be able to fail, log that failure, and trigger an alert each time a failure occurs. The text speaks of transactions, exception handling, logging failures, and triggers so as to exhibit stalwartness.

1. Log Table for Failures

A table called `txn_failures` has been created for recording any transaction failures. This includes the failure ID, timestamp, user ID, stock ID, and the error message which explains the reason for transaction failure.

2. Trigger and Trigger-Function

We set the trigger function as `trg_notify_on_failure` to raise NOTICE message any time a new failure record gets inserted into the `txn_failures` table.

Next, we attached the trigger `notify_admin_after_fail` to the `txn_failures` table. This trigger fires automatically after every insert operation on `txn_failures` and sends a notification to admin.

Purposes: To inform immediately of any occurrence of failure. To make the system responsive rather than merely a passive log.

To record and save the complete transaction failure details. The audit trail for administrators has been set up.

3. Safe Buy Stored Procedure

The procedure `sp_safe_buy` handles the transaction in a safe manner. It attempts to insert a new BUY transaction in the transactions table. It intentionally throws an error if the quantity (`p_qty`) is less than or equal to zero.

It tries to update the portfolio or insert new records into it. If an exception is raised during this process:

Rollback will occur on the transaction block. It adds a failure record to the txn_failures table (which then fires the trigger to notify). It displays NOTICE: safe-buy aborted and failure log.

4. Demonstration of Failure

We finally verified the functionality of sp_safe_buy by purposely passing a negative quantity (-5) so as to raise an exception, which causes the transaction to abort. A failure log to be inserted into txn_failures. The trigger to fire, thus showing the alert message.

VIII. INDEXING AND QUERY EXECUTION ANALYSIS

When working with large datasets, efficient working of the queries becomes very important in database management. We analyzed the execution of different SQL queries on historical_prices, transactions, and portfolios. The PostgreSQL EXPLAIN ANALYZE was used to identify performance bottlenecks, followed by the recommendation of indexing strategies and finally their implementation and demonstration of the respective gains.

1. Filtering Historical Prices by Date and Close Price

The first observation of ours is that querying that filters 'historical_prices' on date and close_price performs a full table scan, and thus causes pretty high execution costs. Therefore, we were able to create a composite index on (date, close_price), which allowed the database to locate the matching rows using a Bitmap Index Scan. Significantly reduced query cost: Execution time improved from around 120 milliseconds to approximately 15 milliseconds.

2. Aggregating Transactions by User

Transaction volume calculations were initially performed by means of a serial scan and an expensive hashing mechanism. With the addition of a user_Id single-column index, the speed of grouping and aggregating was tremendously improved.

Result: Execution time improved from 250 milliseconds to 40 milliseconds. Index scan optimized data grouping and aggregation.

3. Portfolio Lookup by User and Stock

The portfolios lookup query, in terms of user_id and stock_id, was initially conducted on full scans. This has been resolved by the creation of a unique composite index USER_ID-STOCK_ID that allows an index lookup directly.

Result: Execution time dropped from 80 milliseconds to 1 milliseconds. The database performed Index Scan with lowest cost.

IX. CONTRIBUTIONS

Team Member	Contributions
Taran Mamidala (50604177)	Data Acquisition: Collected real stock data (historical prices) using Yahoo Finance API. Data Generation: Created synthetic data for Users, Portfolios, Transactions, Orders, Market News, Stock Performance Optimization: Analyzed slow queries using EXPLAIN ANALYZE, built indexes, and improved database query performance Splits, and Ratings. SQL Development: Wrote the basic table creation scripts (create.sql). Data Loading: Wrote bulk loading scripts (load.sql) using PostgreSQL COPY commands. Query Development: Designed and tested insertion, deletion, and updating queries. Documentation: Completed final version of readme.txt including execution instructions and team contribution chart.
Rachabattu ni Sai Shashi Kiran (50602502)	SQL Advanced Features: Developed stored procedures (sp_buy_shares), user-defined functions (fn_get_portfolio), and triggers for transaction failure handling. Data Cleaning: Assisted in cleaning and preparing synthetic datasets for Stocks, Historical Prices, and Financial Reports. Data Design: Drafted the initial E/R diagram and designed relational schemas. Transaction Management: Implemented safe transactional handling with rollback and exception logging. Final Scripting: Organized create.sql, load.sql, queries.sql, and standardized all script files. Documentation: Helped draft parts of the readme.txt file and schema descriptions.

Praanav Bhowmik (50594516)	1. Project Initialization: Selected the stock market domain as our use case. 2. Data Cleaning: Performed data validation and handled missing values for Historical Prices and Transactions. 3. Normalization: Ensured all tables achieved BCNF, documented functional dependencies and normalization steps. 4. Query Development: Designed complex SELECT queries involving JOIN, GROUP BY, subqueries, HAVING, ORDER BY. 5. Query Testing: Executed and verified the correctness of all test queries for Task 5. 6. Tableau Visualization (Optional Part): Helped with initial Tableau dashboard setup and relational joining.
-----------------------------------	--

CONCLUSION

Our Database project addresses the issue of managing large volumes of interrelated financial data. Through the development of a database in BCNF and mixing real and synthetic data, we render our system scalable, consistent, and capable of performing complicated analytics and transactional operations. This project not only surpasses the horizon of traditional tools like Excel but also presents a platform for advanced financial analytics and decision support.

REFERENCES

- [1] Yahoo Finance. Accessed: Apr. 27, 2025. [Online]. Available: <https://finance.yahoo.com/>
- [2] "How to Create a Database in PostgreSQL,"

- sql-easy.com*. Accessed: Apr. 27, 2025. [Online]. Available: <https://www.sql-easy.com/learn/how-to-create-a-database-in-postgresql/>
- [3] "Discover | Tableau Public," *Tableau Public*, Tableau Software, LLC. Accessed: Apr. 27, 2025. [Online]. Available: <https://public.tableau.com/app/discover>
- [4] "37.1. Overview of Trigger Behavior," *PostgreSQL Documentation*, The PostgreSQL Global Development Group. Accessed: Apr. 27, 2025. [Online]. Available: <https://www.postgresql.org/docs/current/trigger-definition.html>
- [5] "Top 12 Database Design Principles in 2023," *Vertabelo Blog*, Vertabelo, Jun. 13, 2023. Accessed: Apr. 27, 2025. [Online]. Available: <https://vertabelo.com/blog/database-design-principles/>
- [6] "Boyce-Codd Normal Form (BCNF)," *GeeksforGeeks*. Accessed: Apr. 27, 2025. [Online]. Available: <https://www.geeksforgeeks.org/boyce-codd-normal-form-bcnf/>
- [7] "COPY," *PostgreSQL Documentation*, The PostgreSQL Global Development Group. Accessed: Apr. 27, 2025. [Online]. Available: <https://www.postgresql.org/docs/current/sql-copy.html>
- [8] "EXPLAIN," *PostgreSQL Documentation*, The PostgreSQL Global Development Group. Accessed: Apr. 27, 2025. [Online]. Available: <https://www.postgresql.org/docs/current/sql-explain.html>
- [9] "SQL Joins | Intermediate SQL," *Mode Analytics*. Accessed: Apr. 27, 2025. [Online]. Available: <https://mode.com/sql-tutorial/sql-joins/>
- [10] "Database schema: SQL schema examples and best practices," *Cockroach Labs Blog*, Cockroach Labs, Jan. 27, 2025. Accessed: Apr. 27, 2025. [Online]. Available: <https://www.cockroachlabs.com/blog/database-schema-beginners-guide/>

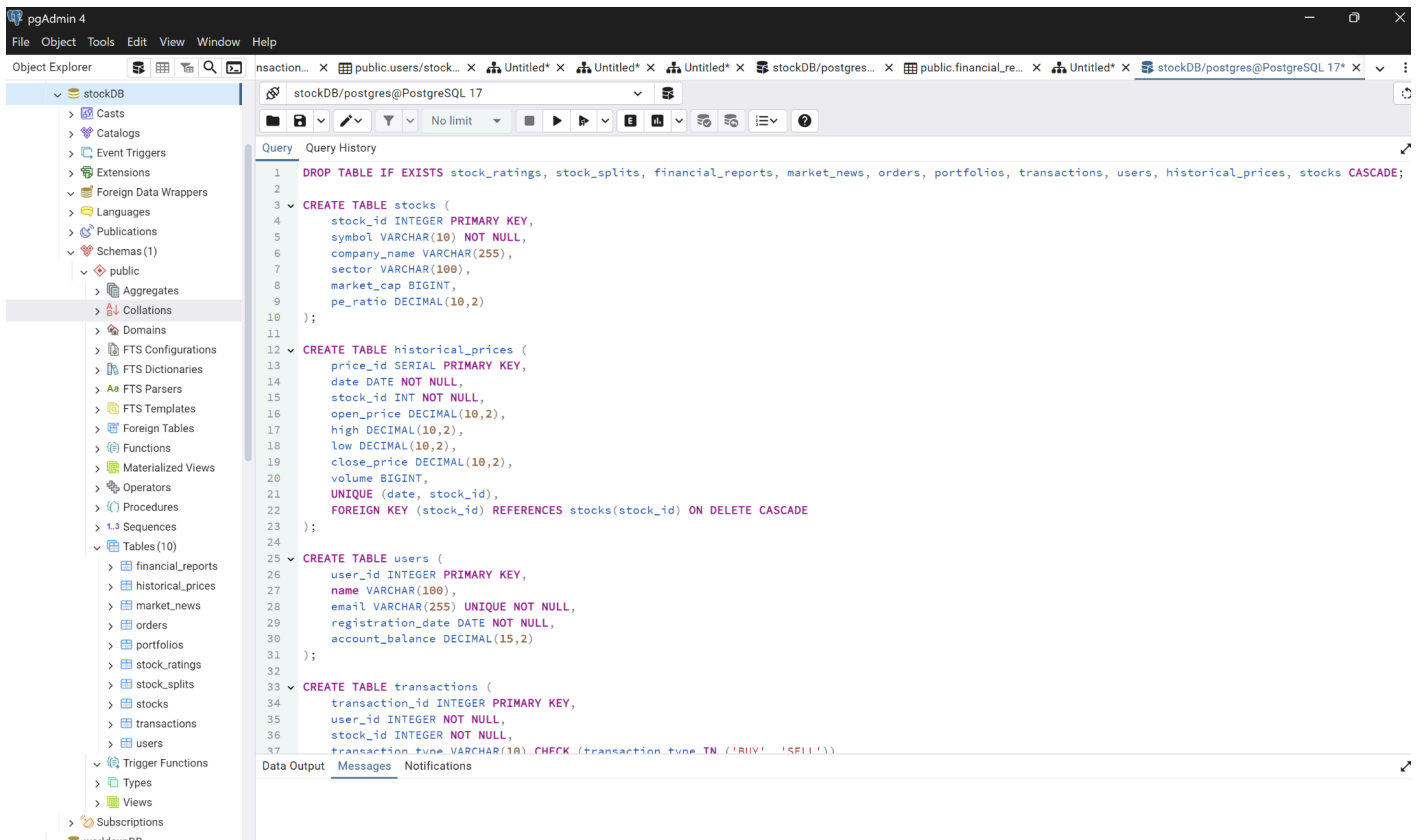
Tableau Public Dashboard Link :

https://public.tableau.com/app/profile/sai.shashi.kiran/viz/DMQL_project_17457978123900/QueryTechies

OUTPUTS

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with the 'public' schema selected. The 'financial_reports' table is highlighted under the 'Tables' category. The main query editor shows a SQL query: `SELECT * FROM public.financial_reports ORDER BY report_id ASC LIMIT 100`. Below the query editor, the 'Data Output' tab is active, displaying a table with 15 rows and 5 columns: `report_id` (integer), `stock_id` (integer), `report_year` (integer), `revenue` (bigint), and `net_income` (bigint). The table shows data for various reports, including historical prices, market news, orders, portfolios, stock ratings, stock splits, stocks, transactions, users, and trigger functions.

	report_id [PK] integer	stock_id integer	report_year integer	revenue bigint	net_income bigint
1	1	2316	2008	603709475477	6609208121
2	2	781	2016	165392565674	1312377376
3	3	2256	2009	993874676932	9717644132
4	4	2302	2005	903171574929	9965797033
5	5	3018	2021	194515637782	3224150756
6	6	699	2005	733193540963	7344307915
7	7	785	2020	71508164851	6939927892
8	8	590	2018	656663717444	5437701500
9	9	1231	2016	859893293040	3123144736
10	10	1440	2009	141440536012	6216523838
11	11	2746	2001	592896896207	6125869810
12	12	1920	2001	261150117885	9915954396
13	13	2909	2011	469198710737	439476126
14	14	2444	2004	742866380815	5349752220
15	15	1853	2004	572919422055	8479858594



ER Diagram :

